Batch 13 — Kitchen + Bar Mobile Screens

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Tue, 16 Jun at 08:14

Batch 13 — Kitchen + Bar Mobile Screens

Meat Lovers CIMS powered by YohPal

13A. Update

apps/pos-pwa/src/app/routes.jsx

Add imports:

```
import KitchenQueuePage from '../features/kitchen/pages/KitchenQueuePage'
import BarStockPage from '../features/bar/pages/BarStockPage'
import BarIssuePage from '../features/bar/pages/BarIssuePage'
```

Add routes inside `PosLayout`:

```
<Route path="/kitchen" element={<KitchenQueuePage />} />
<Route path="/bar-stock" element={<BarStockPage />} />
<Route path="/bar-issue" element={<BarIssuePage />} />
```

13B. Update

apps/pos-pwa/src/components/common/MobileNav.jsx

Replace file with:

```
import React from 'react'
import { Link } from 'react-router-dom'

export default function MobileNav() {
  return (
    <nav
      style={{
        position: 'fixed',
        bottom: 0,
        left: 0,
        right: 0,
        background: '#111827',
        color: '#fff',
        display: 'grid',
        gridTemplateColumns: 'repeat(4, 1fr)',
        gap: 4,
        padding: 10,
        zIndex: 20,
        fontSize: 13,
        textAlign: 'center',
      }}
    >
      <Link to="/menu">POS</Link>
      <Link to="/kitchen">Kitchen</Link>
      <Link to="/bar-stock">Bar</Link>
      <Link to="/orders">Orders</Link>
    </nav>
  )
}
```

13C.

apps/pos-pwa/src/features/kitchen/api/kitchenApi.js

```
import apiClient from '../../../lib/apiClient'

export const kitchenApi = {
  orders: async () => {
    const { data } = await apiClient.get('/kitchen/orders')
```

```
    return data.data.kitchen_orders
  },

  markPreparing: async (orderId) => {
    const { data } = await apiClient.post(`/kitchen/orders/${orderId}/mark-preparing`, {})
    return data
  },

  markReady: async (orderId) => {
    const { data } = await apiClient.post(`/kitchen/orders/${orderId}/mark-ready`, {})
    return data
  },
}
```

13D.

apps/pos-pwa/src/features/bar/api/barApi.js

```
import apiClient from '../../../lib/apiClient'

export const barApi = {
  stock: async () => {
    const { data } = await apiClient.get('/bar/stock')
    return data.data.bar_stock
  },

  issue: async (payload) => {
    const { data } = await apiClient.post('/bar/stock-issue', payload)
    return data
  },
}
```

13E.

apps/pos-pwa/src/features/kitchen/pages/KitchenQueuePage.jsx

```
import React from 'react'
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query'
import { kitchenApi } from '../api/kitchenApi'

export default function KitchenQueuePage() {
  const qc = useQueryClient()

  const { data, isLoading, error } = useQuery({
    queryKey: ['mobile-kitchen-orders'],
    queryFn: kitchenApi.orders,
  })

  const preparingMutation = useMutation({
    mutationFn: kitchenApi.markPreparing,
    onSuccess: () => qc.invalidateQueries({ queryKey: ['mobile-kitchen-orders'] }),
  })

  const readyMutation = useMutation({
    mutationFn: kitchenApi.markReady,
    onSuccess: () => qc.invalidateQueries({ queryKey: ['mobile-kitchen-orders'] }),
  })

  return (
    <main>
      <h1>Kitchen Queue</h1>
      <p>Food production queue for chef/kitchen staff.</p>

      {isLoading ? <p>Loading kitchen orders...</p> : null}
      {error ? <p>Could not load kitchen orders.</p> : null}

      <div style={{ display: 'grid', gap: 12 }}>
        {(data || []).map((order) => (
          <div key={order.id} style={cardStyle}>
            <strong>{order.order_number}</strong>
            <p>Table: {order.table_number}</p>
            <p>Status: {order.order_status}</p>
            <p>Created: {order.created_at}</p>

            <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 8 }}>
              <button
                onClick={() => preparingMutation.mutate(order.id)}
                style={secondaryButton}
              >
                Mark Preparing
              </button>
```

```
              <button
                onClick={() => readyMutation.mutate(order.id)}
                style={primaryButton}
              >
                Mark Ready
              </button>
            </div>
          </div>
        ))}
      </div>
    </main>
  )
}

const cardStyle = {
  background: '#fff',
  padding: 14,
  borderRadius: 14,
  border: '1px solid #e5e7eb',
}

const primaryButton = {
  padding: 12,
  borderRadius: 12,
  border: 'none',
  background: '#111827',
  color: '#fff',
}

const secondaryButton = {
  padding: 12,
  borderRadius: 12,
  border: '1px solid #111827',
  background: '#fff',
  color: '#111827',
}
```

13F.

apps/pos-pwa/src/features/bar/pages/BarStockPage.jsx

```
import React from 'react'
import { Link } from 'react-router-dom'
import { useQuery } from '@tanstack/react-query'
import { barApi } from '../api/barApi'

export default function BarStockPage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['mobile-bar-stock'],
    queryFn: barApi.stock,
  })

  const lowStock = (data || []).filter(
    (item) => Number(item.current_quantity) <= Number(item.reorder_level)
  )

  return (
    <main>
      <h1>Bar Stock</h1>
      <p>Alcoholic drinks accountability and low-stock control.</p>

      <Link to="/bar-issue" style={buttonStyle}>
        Issue Bar Stock
      </Link>

      {lowStock.length > 0 ? (
        <div style={alertStyle}>
          <strong>Low Stock Alert</strong>
          <p>{lowStock.length} bar items are at or below reorder level.</p>
        </div>
      ) : null}

      {isLoading ? <p>Loading bar stock...</p> : null}
      {error ? <p>Could not load bar stock.</p> : null}

      <div style={{ display: 'grid', gap: 12, marginTop: 16 }}>
        {(data || []).map((item) => (
          <div key={item.id} style={cardStyle}>
            <strong>{item.product_name}</strong>
            <p>Quantity: {item.current_quantity}</p>
            <p>Reorder Level: {item.reorder_level}</p>

            {Number(item.current_quantity) <= Number(item.reorder_level) ? (
              <span style={badgeStyle}>LOW STOCK</span>
```

```
            ) : (
              <span style={okBadgeStyle}>OK</span>
            )}
          </div>
        ))}
      </div>
    </main>
  )
}

const cardStyle = {
  background: '#fff',
  padding: 14,
  borderRadius: 14,
  border: '1px solid #e5e7eb',
}

const alertStyle = {
  background: '#fff1f2',
  color: '#be123c',
  border: '1px solid #fecdd3',
  padding: 14,
  borderRadius: 14,
  marginTop: 16,
}

const badgeStyle = {
  display: 'inline-block',
  padding: '5px 10px',
  borderRadius: 999,
  background: '#fee2e2',
  color: '#991b1b',
  fontSize: 12,
}

const okBadgeStyle = {
  display: 'inline-block',
  padding: '5px 10px',
  borderRadius: 999,
  background: '#dcfce7',
  color: '#166534',
  fontSize: 12,
}

const buttonStyle = {
  display: 'inline-block',
  padding: '12px 14px',
  borderRadius: 12,
  border: 'none',
  background: '#111827',
  color: '#fff',
}
```

13G.

apps/pos-pwa/src/features/bar/pages/BarIssuePage.jsx

```
import React, { useState } from 'react'
import { barApi } from '../api/barApi'

export default function BarIssuePage() {
  const [form, setForm] = useState({
    product_id: '',
    quantity: '',
    reference_number: '',
    notes: '',
  })

  const [status, setStatus] = useState('')

  async function submit(e) {
    e.preventDefault()

    try {
      setStatus('Issuing stock...')

      await barApi.issue({
        product_id: Number(form.product_id),
        quantity: Number(form.quantity),
        reference_number: form.reference_number,
        notes: form.notes,
      })

      setStatus('Bar stock issued successfully.')
```

```
      setForm({
        product_id: '',
        quantity: '',
        reference_number: '',
        notes: '',
      })
    } catch (error) {
      setStatus('Could not issue bar stock.')
    }
  }

  return (
    <main>
      <h1>Issue Bar Stock</h1>
      <p>Record bar stock movement for alcoholic drinks.</p>

      {status ? <p>{status}</p> : null}

      <form onSubmit={submit} style={{ display: 'grid', gap: 12 }}>
        <input
          placeholder="Product ID"
          value={form.product_id}
          onChange={(e) => setForm({ ...form, product_id: e.target.value })}
          style={inputStyle}
        />

        <input
          placeholder="Quantity"
          value={form.quantity}
          onChange={(e) => setForm({ ...form, quantity: e.target.value })}
          style={inputStyle}
        />

        <input
          placeholder="Reference Number"
          value={form.reference_number}
          onChange={(e) => setForm({ ...form, reference_number: e.target.value })}
          style={inputStyle}
        />

        <textarea
          placeholder="Notes"
          value={form.notes}
          onChange={(e) => setForm({ ...form, notes: e.target.value })}
          style={inputStyle}
        />

        <button style={buttonStyle}>
          Issue Stock
        </button>
      </form>
    </main>
  )
}

const inputStyle = {
  width: '100%',
  padding: 14,
  borderRadius: 12,
  border: '1px solid #d1d5db',
}

const buttonStyle = {
  padding: 14,
  borderRadius: 12,
  border: 'none',
  background: '#111827',
  color: '#fff',
}
```

13H. Update

backend/app/Controllers/KitchenController.php

Add this method inside the class:

```
public function markPreparing(int $orderId): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CHEF']);

    $stmt = Database::connection()->prepare(
        "UPDATE orders
         SET order_status = 'PREPARING'
         WHERE id = :id"
```

```
    );

    $stmt->execute(['id' => $orderId]);

    AuditLogger::log(
        (int) $user['id'],
        'KITCHEN',
        'MARK_ORDER_PREPARING',
        $orderId
    );

    Response::success([], 'Order marked as preparing');
}
```

13I. Update

backend/routes/api.php

Add this route:

```
$router->post('/api/kitchen/orders/{id}/mark-preparing', [KitchenController::class, 'markPreparing']);
```

13J.

docs/KITCHEN_BAR_MOBILE_SCREENS.md

```
# Batch 13 — Kitchen + Bar Mobile Screens

## System

Meat Lovers CIMS powered by YohPal

## Purpose

This batch extends the mobile POS app so kitchen and bar staff can operate from smartphones.

## Kitchen Screens

### Kitchen Queue

Allows kitchen/chef staff to:

- view pending orders
- view preparing orders
- mark order as preparing
- mark order as ready

## Bar Screens

### Bar Stock

Allows bar staff to:

- view alcoholic drinks stock
- see current quantity
- see reorder level
- receive low-stock alerts

### Bar Issue

Allows bar staff to:

- issue bar stock
- record product ID
- record quantity
- record reference number
- record notes

## Controls

### Kitchen

Kitchen cannot delete orders.

Kitchen can only update status:

- PREPARING
- READY

### Bar

Bar stock movement is recorded through backend API.
```

```
All bar stock issues are audit logged.

## Mobile-First Rule

All pages are designed for smartphones first.

The bottom navigation gives fast access to:

- POS
- Kitchen
- Bar
- Orders
```

Batch 13 Outcome

Kitchen + bar mobile screens now include:

✅ kitchen login-compatible screen
✅ kitchen order queue
✅ mark preparing
✅ mark ready
✅ bar stock screen
✅ bar issue screen
✅ bar low-stock alert
✅ mobile-first kitchen/bar navigation

Next Smart Move

Build **Batch 14 — Cashier Payment Screen + Receipt Flow**, including:

- cashier mobile/payment page
- order search
- cash settlement
- M-Pesa pending placeholder
- paid receipt view
- kitchen/bar receipt print queue placeholder
- payment audit visibility